

MineSight: Vision-Augmented Minecraft System

Updated Technical Specification — June 2026

Overview

MineSight is a real-time AI-powered perception system for Minecraft. It separates responsibilities between a lightweight client-side mod and an external Python-based machine learning engine. The system detects ores and other objects using computer vision and overlays that information back into the game, with support for tracking, memory, and optional 3D world mapping.

Core Concept

- The Minecraft mod does not perform detection
 - The Python backend performs all ML inference
 - Communication is handled via WebSockets
 - The system operates in real-time
-

Architecture

1. Minecraft Client Mod (Forge 1.8.9)

Responsibilities:

- Send player state (position, rotation, FOV)
- Maintain WebSocket connection to Python backend
- Receive detection results
- Render overlays (2D and optionally 3D)

Does NOT:

- Perform any ore detection
- Scan chunks for ores

2. Python ML Engine

Responsibilities:

- Capture screen (Minecraft window)
- Run object detection (YOLO26s)
- Track detected objects across frames
- Send detection results back to mod

Optional:

- Maintain memory of detected objects
- Predict object locations

- Cluster detections into groups (e.g., ore veins)

3. Communication Layer (WebSocket)

- Persistent, bidirectional connection
 - JSON-based protocol
 - Low-latency messaging
-

ML System (Python)

Model

★ **UPDATED: Model upgraded from YOLO (generic) to YOLO26s**

★ **UPDATED: Hardware target: NVIDIA RTX 4060 Ti (personal use, single machine)**

Variant	Use Case	Status
YOLO26n (nano)	Low-end hardware / CPU-only	Fallback option
YOLO26s (small)	RTX 4060 Ti — PRIMARY TARGET	Selected
YOLO26m (medium)	High-end GPU, max accuracy	Optional upgrade

Why YOLO26s for This Project

- YOLO26s has enhanced small-object detection via ProgLoss and STAL loss functions — critical for small/occluded ores
- NMS-free end-to-end architecture reduces post-processing latency
- RTX 4060 Ti runs YOLO26s at 100+ FPS at 640 resolution — well within the 30 FPS target
- Higher accuracy than nano without meaningful performance cost on target hardware
- Pretrained weights available: yolo26s.pt from Ultralytics assets

Recommended Stack

★ **UPDATED: Python 3.10+ (unchanged)**

- PyTorch with CUDA (RTX 4060 Ti — verify with `torch.cuda.is_available()`)
- Ultralytics YOLO26 (pip install ultralytics)
- OpenCV
- mss for screen capture
- websockets for communication

GPU Verification

```
import torch
print(torch.cuda.is_available())      # Must return True
print(torch.cuda.get_device_name(0))  # Should show RTX 4060 Ti
```

Detection Pipeline

1. Capture screen frame (mss)

2. Run inference — YOLO26s at 640px input resolution
3. Extract bounding boxes
4. Assign IDs (tracking)
5. Send results via WebSocket

Performance Notes (RTX 4060 Ti)

★ **UPDATED: Inference FPS cap from doc (10-30 FPS) is conservative — 4060 Ti can run uncapped**

- Run inference at full GPU speed; no artificial FPS limit needed
- Use 640px or 1280px input resolution — both are viable on this hardware
- All optional features (memory, clustering, 3D mapping) can be enabled without hardware concern
- Python backend and Minecraft client run on the same machine simultaneously

Training Specification

★ **UPDATED: This section is new — not present in original spec**

Dataset

Parameter	Value
Total images	2,524
Train split	~2,024 (80%)
Validation split	~250 (10%)
Test split	~250 (10%)
Annotation tool	Roboflow
Export format	YOLO (auto-generates data.yaml)

Data Collection Strategy

- Primary source: Roboflow Universe (existing Minecraft ore datasets)
- Supplement with manual screenshots at varied brightness/gamma (cave lighting variance)
- Intentionally oversample rare ores (diamond, emerald) — rarity in-game causes class imbalance
- Include partially occluded ores — fully exposed examples alone will cause real-world failures
- Capture at multiple FOV values (matches FOV field sent in player state messages)

Roboflow Augmentations (apply before export)

- Brightness/exposure variation — critical for cave vs. surface lighting
- Horizontal flip — free data, ores are symmetrical
- Random crop and zoom — simulates varied viewing distances
- Mosaic augmentation — also supported natively by YOLO26 during training

Training Command

```
from ultralytics import YOLO
```

```
model = YOLO("yolo26s.pt") # Load pretrained YOLO26s weights
```

```
model.train(  
    data="data.yaml",          # Exported from Roboflow  
    epochs=100,  
    imgsz=640,  
    batch=16,                  # Safe for 4060 Ti VRAM  
    device=0,                  # RTX 4060 Ti  
    project="minesight",  
    name="yolo26s_ores",  
)
```

Expected training time on RTX 4060 Ti: ~15-30 minutes for 100 epochs

Tip: Train both yolo26n and yolo26s, compare mAP on validation set before committing

Data Flow

Python → Mod (detections)

```
{  
  "type": "detections",  
  "objects": [  
    {  
      "label": "diamond_ore",  
      "x": 540,  
      "y": 320,  
      "w": 40,  
      "h": 40,  
      "confidence": 0.92,  
      "id": 12  
    }  
  ]  
}
```

Mod → Python (player state)

```
{  
  "type": "player",  
  "x": 123,  
  "y": 64,  
  "z": -20,  
  "yaw": 90,  
  "pitch": -15,  
  "fov": 90  
}
```

Rendering System (Mod)

2D Overlay

- Draw bounding boxes using screen coordinates

- Use `RenderGameOverlayEvent`
- Color-code based on object type or confidence

3D Overlay (Advanced)

- Convert screen-space detections to world-space
 - Use ray projection from camera
 - Render boxes in world using `RenderWorldLastEvent`
 - Note: ray projection depth estimation is noisy — expect positional error, especially at angles
-

Object Tracking

Enhance detections with tracking:

- Assign persistent IDs to objects
- SORT — lightweight, good baseline
- DeepSORT — better re-identification after occlusion

Note: SORT/DeepSORT expect consistent frame rates. If inference runs slower than game render rate, the mod should hold the last known detection state between updates.

Benefits:

- Stable overlays
 - Reduced flickering
 - Enables prediction
-

Memory System (Optional)

Store detected objects across frames:

```
{
  "ore_nodes": [
    {
      "x": 120,
      "y": 11,
      "z": -45,
      "confidence": 0.92,
      "last_seen": 123456789
    }
  ]
}
```

Capabilities:

- Persistent ore locations
 - Rediscovery after occlusion
 - Heatmaps / density maps
-

Screen to World Mapping (Advanced)

Goal: Convert 2D detections into 3D world coordinates

Inputs:

- Player position
- Camera rotation (yaw, pitch)
- Field of view (FOV)
- Screen resolution

Method:

1. Normalize screen coordinates
2. Convert to camera space ray
3. Apply rotation transforms
4. Raycast into world

Output: Approximate 3D position of detected object

This is the most complex component. Depth estimation from 2D bounding box centers introduces real-world error. Flag this in implementation and iterate.

Performance Considerations

Python (RTX 4060 Ti — updated)

★ **UPDATED: Original doc recommended limiting to 10-30 FPS. Not needed on target hardware.**

- Run inference at full GPU speed — no artificial cap needed on 4060 Ti
- Use 640px or 1280px input resolution
- GPU acceleration via CUDA — always enabled on target machine

Mod

- Avoid heavy logic in render loop
- Cache detections per frame
- Batch rendering calls

Network

- Keep messages lightweight
 - Avoid sending images over WebSocket
 - Use JSON or compact binary format
-

Optional Features

Minimap / Radar

- Top-down visualization of detected ores
- Directional indicators

Prediction Layer

- Predict likely ore locations using ML
- Based on patterns (depth, biome, etc.)

Action Suggestions

```
{  
  "type": "suggestion",  
  "action": "turn_left",  
  "reason": "diamond detected"  
}
```

Alerts

- Sound or visual alerts for rare ores
 - Priority highlighting
-

Development Roadmap

Phase 1: Basic Detection

- Python captures screen
- Runs YOLO26s inference
- Sends detections
- Mod renders 2D boxes

Phase 2: Stability

- Add object tracking (SORT or DeepSORT)
- Improve overlay rendering
- Optimize performance

Phase 3: Intelligence

- Add memory system
- Implement clustering (veins)
- Add prioritization

Phase 4: 3D Awareness

- Implement screen to world mapping
- Render 3D markers

Phase 5: Advanced Features

- Radar/minimap
 - Prediction models
 - Action suggestions
-

Notes

- Designed for singleplayer — personal use on a single machine
- Multiplayer servers may restrict or ban modified clients regardless of overlay-only approach
- ML accuracy depends heavily on training data quality and class balance
- YOLO26s pretrained weights: <https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo26s.pt>

Goal: Build a real-time perception system that augments Minecraft gameplay with AI-driven insights, bridging computer vision and 3D world interaction.